Matematikk **R1**

Fagdag høsten 2026

Tema: **Numerisk derivasjon**

Fagstoff: Kapittel 3E læreboka

INNLEDNING OG **TEORI**

Hensikten med dette miniprosjektet er å bruke programmering til numerisk derivasjon. På slutten av dagen vil dere sitte igjen med kunnskap om hvordan man kan derivere ved hjelp av programmering, og hvordan man kan bruke numeriske metoder til å løse problemer.

OPPGAVEBESKRIVELSE OG INFORMASJON OM LEVERING

Oppgaven går ut på å lage et program i Python, og svare på oppgaver knyttet til dette programmet. Besvarelsen skal leveres som et Word-dokument, og tilhørende py-filer hvis lærer ønsker det. Din faglærer informerer om hvor besvarelsen skal leveres.

Du skal ikke bruke KI til å generere kode.

HVORDAN LAGE EN KODE SOM ER LESBAR?

- *Velg variabelnavn som har noe med konteksten å gjøre, og sørg for at koden er så lesbar som mulig.*
- *Bruk # for å legge inn forklaringer etter linjer.*
- *Du kan bruke tre anførselstegn: ''', over og under hvis du skriver flere linjer som ikke skal være en del av koden.*

DEN DERIVERTE OG NEWTONS KOEFFISIENT (s. 143 – 146 i ASCHEHOUG R1)

Definisjonen av den deriverte er gitt ved

$$f'(x) = \lim_{\Delta x \to 0} \frac{f(x+\Delta x) - f(x)}{\Delta x}$$

En datamaskin kan ikke regne ut grenseverdier hvor $\Delta x \to 0$, men kan derimot regne med svært små tall. Vi kan derfor erstatte grenseverdien med at Δx blir er veldig lite tall, og vil ha en tilnærming for den deriverte.

Vi skriver da:

$$f'(a) \approx \frac{f(a+\Delta x) - f(a)}{\Delta x}$$

hvor Δx er et lite tall, og *a* er *x*-verdien til det punktet vi ønsker å finne den deriverte i.

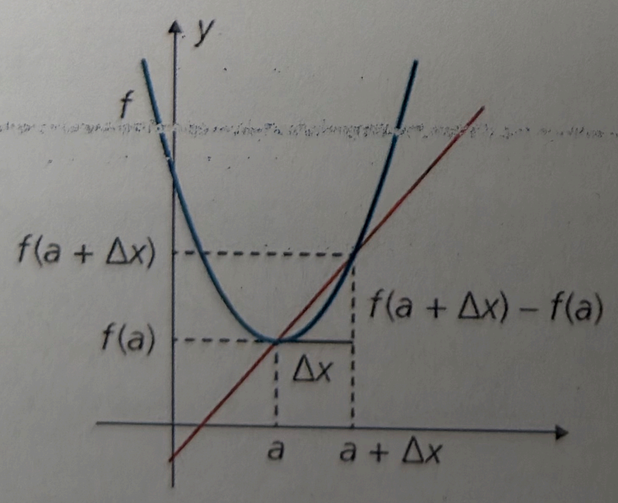
Denne brøken kalles for Newtons kvotient. Telleren i Newtons kvotient kalles en differanse.

Det finnes flere typer differanser. Vi ser på to av dem:

Fremoverdifferanse (i Newtons kvotient): $f(a+\Delta x) - f(a)$

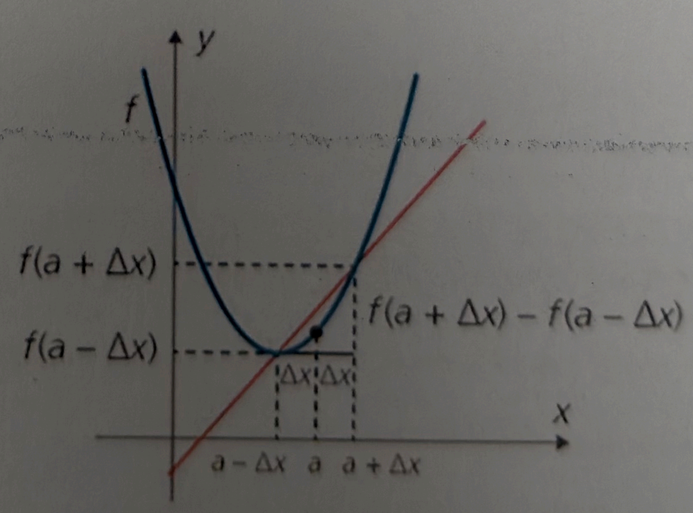
Symmetrisk differanse (i Newtons symmetriske kvotient): $f(a+\Delta x) - f(a-\Delta x)$

Begge disse gir ulike uttrykk for Newtons kvotient, og har forskjellige bruksområder.



$$f'(a) \approx \frac{f(a+\Delta x) - f(a)}{\Delta x}$$

Newtons kvotient



$$f'(a) \approx \frac{f(a+\Delta x) - f(a-\Delta x)}{2\Delta x}$$

Newtons symmetriske kvotient

OPPGAVE A

Dere skal alle jobbe med en tredjegradsfunksjon som er (mer eller mindre) tilfeldig generert av kunstig intelligens. Bruk instruksen under når du ber KI generere polynomet.

> *«Lag et tilfeldig tredjegradspolynom, som har toppunkt og bunnpunkt, og hvor alle koeffisientene er heltallsverdier ulik null. Ikke gi noen begrunnelse eller forklaring.»*

Test funksjonen i Geogebra. Hvis KI feiler, og du får en funksjon som ikke har bunnpunkt og toppunkt, juster på koeffisientene slik at du får det, og skriv hva du har gjort.

OPPGAVE B

Det er kun modulene numpy og matplotlib.pyplot du skal importere til prosjektet.

Under ser du et eksempel på bruk av funksjonen linspace. Her er x_verdier et array, en liste med tall, som blir laget med kommandoen linspace. Denne vil inneholde x-verdier som argument til funksjoner og den deriverte av funksjoner senere i oppgaven. (Et argument er det du putter inn i en funksjon, for eksempel en verdi for x).

```
import numpy as np
import matplotlib.pyplot as plt

# Betingelser for array
xmin = -10                      # Nederste verdi for x
xmax = 10                       # Øverste verdi for x
antall_punkter = 100            # Antall inndelinger
x_verdier = np.linspace (xmin, xmax, antall_punkter ) #lager array

def f(x):                                   # Funksjonen f
  return #skriv inn funksjonen din her

def df(x):                                  # Analytisk deriverte til f
  return #skriv in den analytisk deriverte til funksjonen din her
```

Bruk gjerne kodesnutten over som utgangspunkt. Definer funksjonen din og dens analytisk/algebraisk deriverte som to funksjoner, $f(x)$ og $\mathrm{d}f(x)$.

*Du trenger *numpy* for å bruke linspace, *matplotlib* trenger du for plotting.

OPPGAVE C

Vi skal nå se på den numerisk deriverte: Definer en funksjon som bruker en fremoverkvotient til å tilnærme den deriverte. Funksjonen skal bruke *x*-verdiene hvor vi skal finne den deriverte, og steglengden (Δx). Sett $\Delta x = 10^{-4}$ som et utgangspunkt.

Legg så dette inn i koden din. Se kodesnutt under for inspirasjon.

KODESNUTT (s. 144 - 146 i ASCHEHOUG R1)

```
dx =1E-4

def newton_fwd(x,dx):
    return #Skriv inn funksjonen din her
```

For å sjekke at alt fungerer slik vi har tenkt, skal vi få programmet til å skrive ut følgende tre løsninger for *x* = 2:

- den analytisk deriverte (med d*f*)
- den numerisk deriverte med Newtons kvotient (fremoverdifferanse)
- den numerisk deriverte med Newtons symmetriske kvotient (symmetrisk differanse)

Gir løsningene mening?

OPPGAVE D

Nå skal du lage et plot av den deriverte til funksjonen din ved hjelp av den analytisk deriverte og de numeriske tilnærmingene. Start med Newtons kvotient (fremoverdifferanse). Det kan være lurt å gi grafene forskjellige farger slik at det er lett å se forskjell på dem. Bruk gjerne koden under til inspirasjon til å lage et plot av de to i samme vindu.

```
y_org = f(x_verdier)   # lager liste med beregnede verdier fra f(x)
y_ana = df(x_verdier)   # lager liste med analytisk beregnede verdier for f'(x)
y_fwd = fwd(x_verdier)  # lager liste med numerisk beregnet med fremoverkvot.

# plotter selve grafene og gir dem navn
plt.plot(x_verdier, y_org, label = "Opprinnelig funksjon")
plt.plot(x_verdier, y_ana, label = "Den analytisk deriverte")
plt.plot(x_verdier, y_fwd, label = "Num. der. med fremoverkvotient")

# pynting og info i plot
plt.legend()
plt.grid()
plt.xlabel('x')
plt.ylabel('y')
dxTekst = str(dx)
plt.title('Plot med dx = ' +dxTekst)  # Tar med dx-verdi i tittelen
plt.axhline(y = 0, color = "k")
plt.axvline(x = 0, color = "k")
plt.show()
```

1) Øk steglengden (Δx) til du ser en tydelig forskjell på den analytisk deriverte og den numeriske deriverte. *Legg med et plot av dette i oppgaven.*

2) Hva skjer når Δx blir større? Begrunn svaret.

3) Gjør det samme med Newtons symmetriske kvotient så den blir med i det samme plottet. Ved hvilken Δx blir denne tydelig ulik den analytiske?

OPPGAVE E

Nå skal vi prøve å finne et mål på og sammenlikne unøyaktigheten (feilen) til de to løsningene.

Dette kan for eksempel gjøres med å lage to nye arrays som inneholder differansen mellom den analytiske løsningen og de tilnærmede verdiene vi fikk ved bruk av fremoverkvotienten og den symmetriske kvotienten. Husk at vanlige regneoperasjoner fungerer på arrays.

Du kan deretter summere opp absoluttverdien til disse differansene i to nye variable med eksempelvis funksjonen sum() eller en løkke som går gjennom elementene i arrayene. Bruk funksjonen abs() for å ta absoluttverdiene før du summerer (*hvorfor gjør vi det?*). Bruk gjerne koden nedenfor til inspirasjon.

```
diffANK  = y_ana - y_fwd    # lager liste med differanse mellom analytisk og fremoverkvotient
diffANSK = y_ana - y_sym    # lager liste med differanse mellom analytisk og sym. kvotient

feilANK  = 0      #her summeres absoluttverdien av avviket mellom analytisk og fremoverkvotient
feilANSK = 0      #her summeres absoluttverdien av avviket mellom analytisk og sym. kvotient

for i in range(len(diffANK)):                    #len gir oss lengden av lista
    feilANK  = feilANK + (abs(diffANK[i]))
    feilANSK = feilANSK + (abs(diffANSK[i]))
```

OPPGAVE F

Kjør programmet med $\Delta x = 10$, $\Delta x = 10^{-5}$ og $\Delta x = 10^{-15}$. Fra definisjonen til den deriverte skulle man tro at feilen blir mindre jo mindre Δx blir.

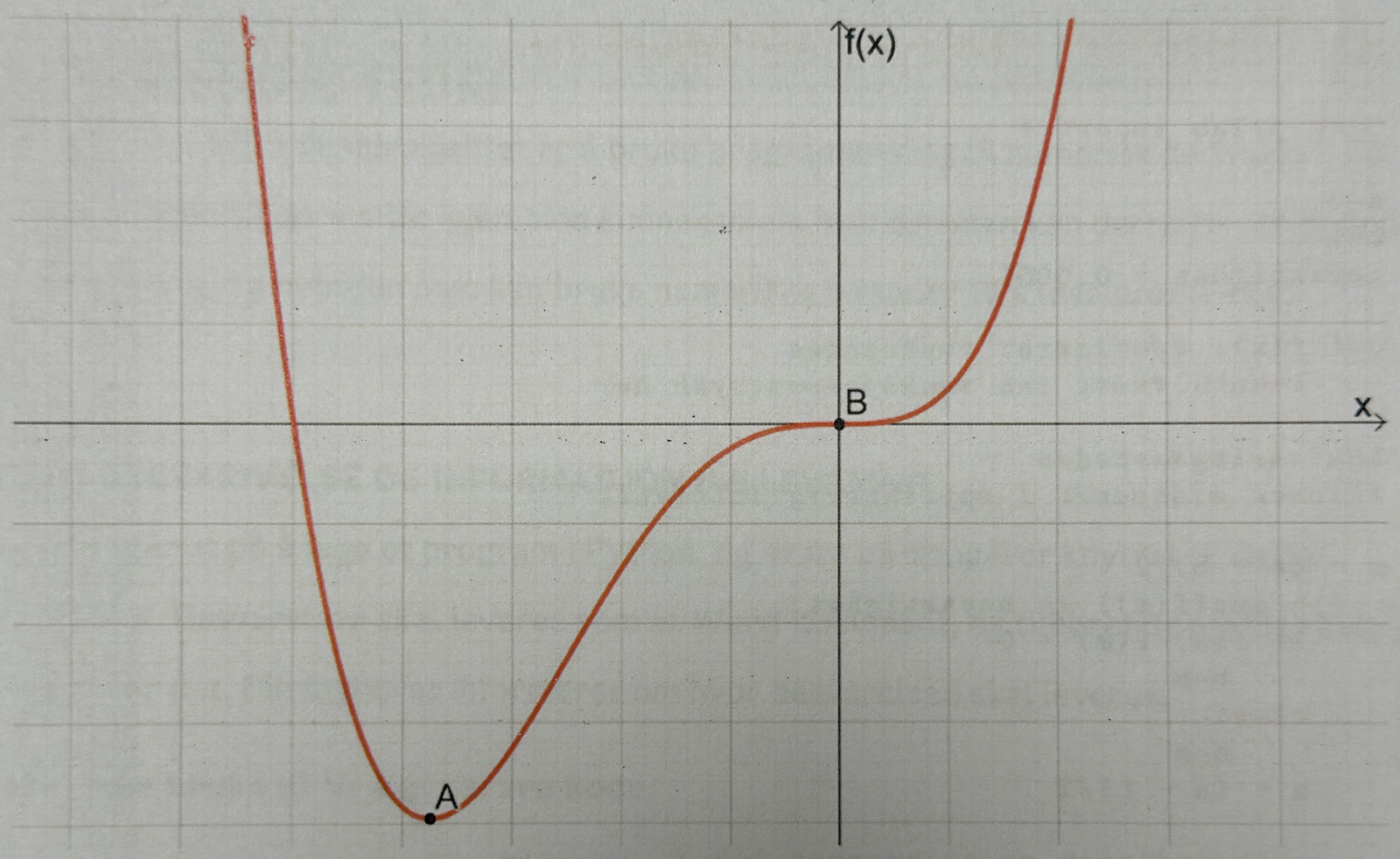
Hvorfor er ikke dette tilfellet?

Det er opp til deg om du velger å lage en liste med verdier for Δx, om du velger å endre Δx mellom hver kjøring av programmet, eller om du velger å la programmet be om verdiene til Δx når det startes.

OPPGAVE G

Det finnes 3 typer stasjonære punkter:

- Toppunkter - den deriverte går fra å være positiv til negativ i det vi går gjennom punktet.
- Bunnpunkter - den deriverte går fra å være negativ til positiv i det vi går gjennom punktet.
- Terrassepunkter - den deriverte har samme fortegn på hver side av det stasjonære punktet.



Figur 1: Grafen til en funksjon med stasjonære punkter

Se på grafen til funksjonen i Figur 1. Bruk fortegnet til den deriverte til å argumentere hvorfor det er et bunnpunkt i punkt A, og et terrassepunkt i punkt B.

Hint: Lag fortegnslinje eller skisser grafen til den deriverte.

OPPGAVE H

Utvid koden slik at den bruker fremoverkvotienten til å finne de stasjonære punktene til funksjonen din, f, fra starten av oppgaven.
Gå gjennom de tilnærmede verdiene for den deriverte og bruk fortegnet til å klassifisere de stasjonære punktene. Programmet skal skrive punktene ut i terminalen.

Hint: Husk at den deriverte er lik 0 i disse punktene. Det betyr at vi kan bruke halveringsmetoden til å finne nullpunktene til den deriverte.

HALVERINGSMETODE KODE (s. 43 i ASCHEHOUG R1)

```
from pylab import *

a=0
b=1
noyaktighet = 0.0001

def f(x): #definerer funksjonen
    return #sett inn funksjonsuttrykk her

#Halveringsmetoden
#Finner midtpunkt i opprinnelig intervall

m = (a + b)/2
while abs(f(m)) >= noyaktighet:
    if f(a)*f(m) < 0:
        b=m
    else:
        a=m
    m = (a + b)/2

print("Loesningen er tilnaermet lik ", round(m, 4)
```

OPPGAVE I

Funksjonen

$$g(x) = 3x^4 + 6x^3$$

inneholder blant annet et terrassepunkt.

1) Utvid programmet ditt fra deloppgave **g)** slik at det også kan klassifisere denne type punkter.
2) Kjør programmet ditt med funksjonen $g(x)$, og skriv de stasjonære punktene ut i terminalen.